

UniVerse

A Campus Companion

Admin & Timetable Engine

The admin control centre and the project's differentiator: an automatic, clash-free timetable generator (OR-Tools CP-SAT).

Leading University, Sylhet - Dept. of CSE

Course CSE-3240 (Project I) - Team Sherlocked

Flutter (Dart) - Supabase - Firebase FCM - OR-Tools CP-SAT

Owner: Fahmid (admin + engine)

Key files: features/admin/*, engine/*.py

1. The admin side

Admins get a dashboard plus management hubs. Each is a screen -> controller -> service slice over Supabase with admin-only RLS.

screen	purpose
Routine hub	Segmented control: Manage (CRUD on routines) + Generate (the engine).
Campus Broadcast	Compose a notification to a chosen audience (+ Broadcast History).
Manage Users	View/manage profiles; admin provisioning via the invite-admin Edge Function.
Manage Resources	Upload files / links (+ Resource Library to delete).
Timetable config	Manage Rooms, Manage Faculty, Timetable Settings - the engine's inputs.

2. The differentiator - why it matters

Building a clash-free routine for ~55 cohorts, ~76 teachers and limited rooms is normally days of manual work. UniVerse does it in seconds with a constraint solver, guarantees no teacher/room/cohort double-booking, and publishes to every student and teacher with one tap. This is the advisor-required feature and the centrepiece of the defense.

3. The pipeline (Excel in -> workbook out)

- Admin picks the Main Distribution .xlsx; the app loads engine config from the DB.
- ingest.py** parses the 'Course Distribution' sheet: binds columns by header text, applies the even-digit lab rule, excludes blank-teacher/blank-batch rows, detects service/non-CSE offerings, validates invariants.
- solver.py Phase 1 (CP-SAT)**: assign each session a (day, period). HARD constraints: each session once; no teacher/cohort/room double-book; teacher day-offs; Friday no period-4; rooms <= pool. SOFT: spread a course's 2 sessions across different days, cohort compactness, avoid the last period.
- solver.py Phase 2 (greedy room assign)**: labs -> lab rooms, theory -> theory/galleries. Then it self-validates.
- render.py** clones the Excel template and writes one canonical 55-row cohort map to all 7 day-sheets, cells as 'CODE TEACHER ROOM'.
- App polls status -> shows report + validation + grid -> Download .xlsx or Publish.

CP-SAT (Constraint Programming - SATisfiability) from Google OR-Tools explores the assignment space far faster than brute force and returns a provably clash-free solution (or reports infeasibility).

4. The HTTP contract (FastAPI engine)

endpoint	contract
POST /generate	multipart {file, config(JSON), time_limit_s} -> {job_id}. Runs on a background thread.
GET /status/{id}	{state, progress, [stats, validation, error]}. state: queued->ingesting->solving->rendering->done failed.
GET /result/{id}	{rows, stats, validation, report}.
GET /download/{id}	the rendered .xlsx.

The engine is **stateless** - jobs live in an in-memory dict, no DB on the engine side. Deployed on Render

free tier; it sleeps after ~15 min so the first request is a ~50s cold start (pre-warm before a demo).

5. Config flow (DB-backed) - keep three in sync

Admin edits Manage Rooms / Faculty / Settings -> ``TimetableConfigService`` CRUDs the ``timetable_*`` tables -> ``buildEngineConfig()`` assembles `{rooms[], teachers[], settings{periods, friday_no_p4, service_scope, weights, semester_label}}` -> sent with the generate request -> the engine's ``_normalize_config()`` consumes it. **If you rename a column, all three must move together** - a key project guardrail.

6. Generate + publish (timetable_gen_controller)

- 1 ``generate()`` builds config, uploads the file, then polls every 2s (up to ~4 min) until the job is done.
- 2 On done it fetches the result (rows + stats + validation + report) for the preview grid.
- 3 ``downloadAndOpen()`` fetches the .xlsx, opens it, and archives it to the ``timetables`` bucket.
- 4 ``publish()`` writes rows into ``routines`` (idempotent: clears each generated batch first), records the run in ``timetable_runs``, then fires a campus-wide 'routine published' notification (in-app + push).

Critical guardrail: the engine emits rows shaped EXACTLY like the ``routines`` columns (`RoutineEntry.toMap()`), so publishing is a direct insert. Change one side and publish silently breaks.

7. Likely defense questions

question	answer
Why CP-SAT / OR-Tools?	Timetabling is a constraint-satisfaction problem; CP-SAT finds a provably clash-free schedule far faster than brute force.
Hard vs soft constraints?	Hard = must hold (no double-booking, day-offs); soft = preferences the solver optimises (spread sessions, compactness).
What proves it's clash-free?	The solver self-validates (<code>validation.ok</code>) and the HARD constraints make double-booking impossible by construction.
Where does it run?	A separate FastAPI service on Render; the app talks to it over HTTPS and polls the job.
Real numbers?	Summer 2025: 358 offerings -> 654 CSE + 32 service sessions, 55 cohorts, 76 teachers, solved clash-free.